

TREE 1

PROBLEM 6

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 66
Question description A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x's boss k levels higher up in the hierarchy? Input The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \dots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \dots, e_n$: for each employee $2, 3, \dots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k: who is employee x's boss k levels higher up? Output Print the answer for each query. If such a boss does not exist, print -1. Constraints $1 \leq n, q \leq 2 \cdot 10^5$ $1 \leq e_i \leq i-1$ $1 \leq x \leq n$ $1 \leq k \leq n$					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>5 3 1 3 2 2 4 2 1 2 1 3</pre> EXPECTED OUTPUT <pre>1 -1 -1</pre> Test Case 2 INPUT (STDIN) <pre>5 3 1 1 3 3 4 1 4 2 4 3</pre> EXPECTED OUTPUT <pre>3 1 -1</pre>					
Mandatory Test Cases Test Case 1 KEYWORD <pre>void link(int i,int j)</pre>					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 2 Test Case 2 TOKEN COUNT 420 Test Case 3 NLOC 50					

SOURCE CODE:

```
#include <stdio.h>

int parent_table[200005][20];

int main() {
    int employee_count, query_count;
    if (scanf("%d %d", &employee_count, &query_count) != 2) return 0;

    parent_table[1][0] = 0;
    for (int emp = 2; emp <= employee_count; emp++) {
        int supervisor;
        scanf("%d", &supervisor);
        parent_table[emp][0] = supervisor;
    }

    for (int power = 1; power < 20; power++) {
        for (int emp = 1; emp <= employee_count; emp++) {
            int mid_ancestor = parent_table[emp][power - 1];
            if (mid_ancestor != 0) {
                parent_table[emp][power] = parent_table[mid_ancestor][power - 1];
            } else {
                parent_table[emp][power] = 0;
            }
        }
    }

    while (query_count--) {
        int target_emp, steps_up;
        scanf("%d %d", &target_emp, &steps_up);

        for (int bit = 19; bit >= 0; bit--) {
            if (steps_up & (1 << bit)) {
                target_emp = parent_table[target_emp][bit];
                if (target_emp == 0) break;
            }
        }
        printf("%d\n", (target_emp == 0) ? -1 : target_emp);
    }
    return 0;
}
```

OUTPUT:

```
5 3
1 1 3 3
4 1
4 2
4 3
3
1
-1
```

```
5 3
1 3 2 2
4 2
1 2
1 3
1
-1
-1
```